

09. 감염관리실 — 감염관리 임상지원 챗봇

공유 게이트웨이(:11500) 위에서 내 앱을 운영급으로. 이 가이드 한 장이면 됩니다. URL은 이미 게이트웨이로 바뀌어 있으니, 살아있는지 확인하고 한 번 돌려보는 것이 오늘 할 일입니다.

1) 내 앱 한눈에

- **아키타입**: A형 — 근거기반(RAG) 챗봇. 사내 감염관리 지침 8조각(IC-01~IC-08: 결핵·중심정맥관·손위생·격리·소독멸균·다제내성균)을 임베딩으로 검색해서 **출처와 함께** 답합니다.
- **핵심기능**: 하네스 모드(검색+출처표시)와 바이브 모드(검색 없이 자유답변)를 토글로 비교. “몇 개/가장 짧은” 같은 개수·비교 질문은 코드로 직접 계산(CALC 힌트)해 헛답을 막고, **환자 식별정보(PHI)가 들어오면 모델 호출 없이 즉시 차단**합니다.
- **구성**: `index.html` (프론트) + `app.py` (FastAPI 백엔드) + `compare.py` (두 모드 비교 스크립트). 모델 호출은 전부 백엔드를 거칩니다(브라우저가 직접 모델을 부르지 않음).

2) 바뀌는 것 — URL 한 줄

이미 게이트웨이로 설정되어 있습니다. 따로 고칠 것이 없습니다.

`app.py` 9번째 줄, 모델 주소가 이미 `:11500` (게이트웨이)을 가리킵니다. 이 줄 **한 줄**이 임베딩·생존확인·채팅 **세 가지 호출 전부**를 한꺼번에 제어합니다.

```
# app.py:9 (현재 — 이미 게이트웨이)
OLLAMA = "http://121.138.151.6:11500"
```

이 한 줄이 아래 세 곳에서 그대로 쓰입니다(주소를 바꿀 곳이 여기 하나뿐이라는 뜻):

```
# app.py:67 임베딩 호출 → f"{OLLAMA}/api/embeddings"
# app.py:152 생존 확인 → f"{OLLAMA}/api/tags"
# app.py:245 채팅 스트리밍 → f"{OLLAMA}/api/chat"
```

원래 어디였나 — 처음 제출 코드는 `:11434` (각자 띄우던 raw 주소)였습니다. 백업 파일 `app.py.bak` 에 원본이 그대로 남아 있어 비교할 수 있습니다:

```
# app.py:9
- OLLAMA = "http://121.138.151.6:11434" # 예전: raw 주소
(생각 누출·부하 위험)
+ OLLAMA = "http://121.138.151.6:11500" # 지금: 공유 게이트웨이
```

만약 어떤 이유로 9번째 줄이 다시 `:11434` 로 보이면, 위 diff처럼 `11434` 를 `11500` 으로 한 글자만 바꾸면 됩니다. 그 외에는 손댈 코드가 없습니다.

3) 확인 절차

(1) 게이트웨이가 살아있나 — curl 로 점검

```
curl http://121.138.151.6:11500/api/tags
```

모델 목록(JSON)이 돌아오면 게이트웨이 정상입니다. 이 앱은 이 `/api/tags` 응답으로 생존을 확인합니다.

(2) 앱 기동 — 정확한 실행 명령

감염관리 폴더로 가서 백엔드를 띄웁니다(포트 8000 고정 — 프론트가 8000을 바라봅니다).

```
cd 내-감염관리-앱-폴더 # app.py 가 들어 있는 본인 폴더로 이동
uvicorn app:app --port 8000
```

그다음 프론트를 띄웁니다(터미널 2). `index.html` 을 파일로 직접 더블클릭하면 브라우저 보안(CORS)으로 막힐 수 있으니, 같은 폴더에서 간단한 웹서버로 여세요:

```
cd 내-감염관리-앱-폴더
python -m http.server 5500
```

브라우저에서 `http://127.0.0.1:5500/index.html` 을 엽니다. 화면 위 상태배치가 초록(정상)이 되는지 보세요 — 배치는 5초마다 `/health` 를 폴링합니다. 앱 자체 상태는 어떻게도 확인할 수 있습니다:

```
curl http://127.0.0.1:8000/health
# {"ok":true,"ollama":true,"model":"qwen3.6:35b-
a3b","docs":8,"top_k":2,...}
```

`"ollama":true` 면 게이트웨이까지 연결 정상입니다.

(3) 대표 질문 1개 — 한번 돌려보기

화면 입력창에 아래를 넣어 보세요(하네스 모드 기본):

활동성 폐결핵 격리해제 조건이 뭐야?

기대 동작: 답 위에 출처 태그 **IC-02 | 결핵관리지침 p.118** 가 먼저 뜨고, 이어서 “항결핵제 2주 이상 + 임상 증상 호전 + 객담도말 연속 3회 음전” 같은 근거 답이 한 글자씩 흘러나옵니다.

명령줄에서 바로 확인하려면:

```
curl -N -X POST http://127.0.0.1:8000/api/chat \
-H "Content-Type: application/json" \
-d '{"question":"활동성 폐결핵 격리해제 조건이 뭐야?"}'
```

(4) think(생각) 누출 없는지 확인

답변 안에 `<think>` 같은 태그나 “사용자가 ~을 물었으니 먼저...” 같은 **모델의 혼잣말(추론 과정)**이 섞여 나오면 안 됩니다. 게이트웨이가 `think:false` 를 자동으로 걸어주므로, 깔끔한 답만 나오는 것이 정상입니다. (참고로 이 앱은 자체적으로 `<think>...</think>` 태그도 한 번 더 지웁니다 — [app.py:262](#).) 만약 혼잣말이 보이면 9번째 줄 주소가 `:11500` 인지부터 다시 확인하세요.

4) 개선 1~2가지 (분석 근거)

1. **모델명도 게이트웨이에 맡기기 (가장 중요)** — [app.py:10](#) 의 `CHAT_MODEL = "qwen3.6:35b-a3b"`, [app.py:11](#) 의 `EMBED_MODEL = "bge-m3:latest"` 가 코드에 하드코딩되어 있습니다. 게이트웨이가 제공하는 모델명과 글자가 다르면 답이 안 옵니다. 게이트웨이는 모델명 정규화를 자동으로 해주니 대개 그대로 동작하지만, **임베딩 모델(bge-m3)**이 게이트웨이에 없으면 하네스(검색) 모드 자체가 깨집니다. 한 번 띄운 뒤 `/health` 의 `"ollama":true` 와 첫 질문에서 출처 태그가 실제로 뜨는지로 확인하세요. 안 뜨면 임베딩 모델명을 게이트웨이 제공명으로 맞추면 됩니다.
2. **think 누출은 게이트웨이가 책임, 코드 정규식은 보조** — 현재 코드는 인라인 `<think>` 태그만 정규식으로 지웁니다([app.py:262](#)). 스트림이 청크 경계에서 태그를 반으로 자르면 놓칠 수 있습니다. 이제는 게이트웨이가 `think:false` 로 추론 자체를 끄므로 이 위험이 사라졌습니다 — **앱 코드를 더 손댈 필요 없이** 게이트웨이로 주소를 둔 것만으로 해결됩니다. (이것이 4주차의 핵심: 앱마다 박던 방어를 게이트웨이 한 곳으로.)

보너스: 답 길이가 들쭉날쭉하면 게이트웨이가 `num_predict` 상한을 자동으로 걸어 응답 시간을 안정시킵니다. 학생이 신경 쓸 것은 없습니다.

5) 안전수칙 (감염관리 특화)

- **환자 식별정보(PHI) 절대 입력 금지** — 환자명·등록번호·주민번호·“○○호 △△△”(병실+이름)는 넣지 마세요. 이 앱은 그런 입력을 감지하면 모델 호출 없이 차단([app.py:166-201](#))하지만, **방어는 보조일 뿐** 입력 자체를 안 하는 것이 원칙입니다. 질문은 항상 질병·지침 내용만(예: “MRSA 격리 해제 기준”).
- **공유 GPU는 1개, 동시성 주의** — 게이트웨이 뒤의 GPU는 모두가 함께 씁니다. 여러 명이 동시에 긴 질문을 던지면 느려집니다. 실습 때는 한 번에 한 질문씩, 답이 끝나면 다음 질문을.
- **대외비·실데이터 외부 반출 금지** — 이 앱의 지침 8조각은 공개 요약문이라 민감도가 낮지만, 실제 사례·내부 통계는 더미(비식별) 데이터로만 다루세요.
- **공유 서버 = 운영 책임** — 인증 없는 공유 환경입니다. 테스트가 끝나면 띄운 `uvicorn` 을 내려(Ctrl+C) 불필요한 부하를 남기지 마세요.